FORGEGIT – GIT AS A SERVICE HOSTED ON MICROSOFT AZURE

CLOUD COMPUTING

Submitted by

SMITH N S(230701323)
SOWNDARIYA R(230701329)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE

NOVEMBER 2025

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled "FORGEGIT – GIT AS A SERVICE HOSTED ON MICROSOFT AZURE" is the bonafide work of SMITH(230701323),SOWNDARIYA R(230701329) who carried out the work under my supervision.

Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. E. M Malathy
Professor & Supervisor
Department of Computer Science and Engineering
Rajalakshmi Engineering College

SIGNATURE

Dr. M. Ayyadurai
Head of The Department
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on _____

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman, Vice Chairman, and Chairperson for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to the Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide for his valuable guidance throughout the course of the project.

SMITH N S
SOWNDARIYA R

ABSTRACT

Modern software development teams and academic institutions increasingly depend on version control systems to manage source code, collaborate across distributed teams, and track changes over time. Existing Git hosting solutions such as GitHub and GitLab, while feature-rich, often involve external data hosting, limited customization, and recurring subscription costs. There is a clear need for a self-hosted, cost-efficient, and fully controlled Git service that educational institutions and small development teams can deploy on their own cloud infrastructure.

ForgeGit is a cloud-hosted Git-as-a-Service platform built by deploying Gitea, a lightweight self-hosted Git service, on Microsoft Azure infrastructure. The system provides a complete web-based interface for repository management, user authentication, issue tracking, code review, and team collaboration. The deployment leverages an Azure Virtual Machine running Ubuntu Server to host the Gitea application, with Azure Virtual Network and Network Security Groups ensuring controlled, secure traffic management, and Azure Network Watcher providing comprehensive network monitoring and diagnostics.

The system supports multiple user roles including administrators, organization owners, and individual developers. Users can create and manage Git repositories, collaborate through pull requests and code reviews, track issues and milestones, and manage organization-level teams and permissions. Gitea's built-in SQLite or PostgreSQL database manages all metadata while Git repositories are stored on the VM's attached disk with regular backup strategies.

The Azure infrastructure includes a Standard B2s Virtual Machine, Virtual Network with dedicated subnets, Network Security Groups with strict inbound and outbound rules, and Azure Network Watcher for ongoing monitoring and traffic flow analysis. The total deployment cost targets under Rs. 2,500 per month, making it viable for academic and small-team deployments. The project demonstrates practical cloud deployment, network security configuration, and real-world DevOps infrastructure management.

TABLE OF CONTENTS

Chapter No. / Title	Page No.
1 Introduction	1
1.1 Problem Statement	2
1.2 Objective of the Project	4
1.3 Scope and Boundaries	6
1.4 Stakeholders and End Users	8
1.5 Technologies Used	10
1.6 Organization of the Report	12
2 System Design and Architecture	14
2.1 Requirement Summary (Functional & Non-Functional)	15
2.2 Proposed Solution Overview	17
2.3 Cloud Deployment Strategy	19
2.4 Infrastructure Requirements	21
2.5 Azure Services Mapping and Justification	23
3 Implementation	26
3.1 Gitea Installation and Configuration Module	27
3.2 User and Organization Management Module	29
3.3 Repository Management Module	31
3.4 Issue and Milestone Tracking Module	33
3.5 Webhook and CI Integration Module	35
3.6 Backup and Storage Module	37
4 Cloud Operations and Security	38
4.1 Azure VM Deployment Configuration	39
4.2 Network Security and NSG Rules	41
4.3 Azure Network Watcher Monitoring	43
4.4 Application Security Features	45
5 Results and Discussion	48
5.1 Implementation Summary	49
5.2 Challenges Faced and Resolutions	51
5.3 Performance Observations	53

5.4 Key Learnings and Team Contributions	55
6 Conclusion and Future Enhancement	58
6.1 Conclusion	59
6.2 Future Scope	61
References	63
Appendices	65

CHAPTER 1 - INTRODUCTION

1.1 PROBLEM STATEMENT

Software development teams, academic institutions, and individual developers rely heavily on version control systems to manage code repositories, track changes, and enable team collaboration. While platforms like GitHub and GitLab provide comprehensive Git hosting, they present significant limitations for institutions requiring data sovereignty, cost control, and platform customization:

- External Data Hosting Risks: Code and intellectual property stored on third-party servers creates data governance challenges, especially for academic institutions and organizations with privacy requirements.
- Subscription Costs: Commercial Git platforms impose per-user pricing that becomes prohibitive for large teams or institutions with limited budgets.
- Limited Customization: Hosted platforms restrict branding, workflow customization, and integration with institutional systems, preventing tailored development environments.
- Dependency on External Availability: Reliance on external providers exposes teams to outages, policy changes, or service discontinuations that disrupt development workflows.
- No Infrastructure Learning: Using hosted platforms provides no exposure to cloud deployment, network security configuration, or DevOps infrastructure management — critical learning outcomes for engineering students.
- Network Access Control Challenges: Organizations often need fine-grained control over who can access their Git infrastructure, which is not easily achievable with public hosted solutions.

These challenges collectively demonstrate the need for a self-hosted, cloud-deployable Git service that provides full control over infrastructure, data, and configuration while remaining cost-efficient and accessible.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of this project is to deploy and configure ForgeGit — a fully functional Git-as-a-Service platform — using Gitea on Microsoft Azure infrastructure. The specific objectives include:

- Deploy a Self-Hosted Git Service: Install and configure Gitea on an Azure Virtual Machine to provide a complete web-based Git hosting platform with repository management, user accounts, and collaboration features.
- Implement Secure Cloud Networking: Configure Azure Virtual Network, subnets, and Network Security Groups to enforce strict traffic control, allowing only necessary ports while blocking unauthorized access.
- Enable Comprehensive Network Monitoring: Utilize Azure Network Watcher for continuous network diagnostics, connection monitoring, and traffic flow analysis to maintain infrastructure health and security visibility.
- Provide Multi-User Repository Management: Support multiple users, organizations, and teams with role-based permissions for repository access, enabling team-based development workflows.

- Integrate Webhook and Automation Support: Configure Gitea webhooks to support integration with CI/CD pipelines and external automation tools, enabling modern DevOps workflows.
- Achieve Cost-Efficient Deployment: Leverage Azure Student Subscription to host the complete Git service within a monthly budget of Rs. 2,000-3,000, demonstrating viable academic cloud deployment.

1.3 SCOPE AND BOUNDARIES

The scope of this project encompasses the complete deployment and configuration of a Gitea-based Git hosting platform on Microsoft Azure. The system provides full Git repository hosting including push/pull operations over HTTPS and SSH, web-based repository browsing, issue tracking, pull request workflows, user and organization management, and webhook-based event notifications.

Infrastructure deployment is carried out on an Azure Linux Virtual Machine configured with a Virtual Network, Subnets, Public IP, Network Security Groups, and Azure Network Watcher for monitoring and traffic analysis. The service is accessible via web browser and Git command-line tools from any device with internet connectivity.

The scope does not include native mobile application development, built-in CI/CD pipelines (Gitea Actions configuration is identified as a future enhancement), or managed database services. These are planned as future scope items.

1.4 STAKEHOLDERS AND END USERS

Stakeholders: Project Developers / Team Members — Design, deploy, and configure all infrastructure components including Gitea setup, Azure networking, NSG rules, and Network Watcher integration.

End Users:

- Developers who require self-hosted repository management, code review via pull requests, and issue tracking for their projects.
- Organization Administrators who manage team permissions, repository access, and user accounts across projects.
- Students and Academic Teams who need a version-controlled collaboration environment for course projects and team assignments.

1.5 TECHNOLOGIES USED

Table 1. Application Technologies

Technology	Purpose
Gitea	Lightweight self-hosted Git service providing web UI, repository management, user accounts, issues, and pull requests
SQLite / PostgreSQL	Embedded database for Gitea metadata including users, repositories, issues, and access tokens
Nginx	Reverse proxy forwarding HTTPS traffic to the Gitea application running on localhost

Git (CLI)	Core version control engine used by Gitea for all repository operations

Table 2. Cloud and Infrastructure Technologies

Service	Purpose
Microsoft Azure	Primary cloud provider hosting all infrastructure components
Azure Virtual Machine (B2s)	Hosts Gitea application, Nginx reverse proxy, and SQLite/PostgreSQL database
Azure VNet & NSG	Secure networking, traffic filtering, and controlled access to Git services
Azure Network Watcher	Network monitoring, diagnostics, connection troubleshooting, and traffic analysis

1.6 ORGANIZATION OF THE REPORT

Chapter 1 introduces the problem context, project objectives, scope, stakeholders, technologies used, and the report structure.

Chapter 2 presents a detailed examination of the system design and architecture, including functional and non-functional requirements, proposed solution overview, cloud deployment strategy, infrastructure specifications, and Azure service mappings.

Chapter 3 covers the implementation details of each module including Gitea installation, user and organization management, repository operations, issue tracking, webhook integration, and backup configuration.

Chapter 4 focuses on cloud operations and security, covering Azure VM deployment configuration, Network Security Group rules, Azure Network Watcher monitoring, and application-level security features.

Chapter 5 presents the results of the implementation including summaries, challenges faced and their resolutions, performance observations, and key learnings.

Chapter 6 concludes the report with a summary of achievements and a roadmap for future enhancements.

CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements

The ForgeGit system consists of cloud-based components deployed on Microsoft Azure to support a secure, scalable, and feature-complete Git hosting environment.

The core application is Gitea, an open-source, self-hosted Git service written in Go. Gitea provides a complete web interface for repository management, user authentication, team collaboration, issue tracking, and pull request workflows. It is hosted on an Azure Linux Virtual Machine and served via Nginx as a reverse proxy.

Repository data and Git objects are stored on the VM's attached SSD disk with a structured directory hierarchy maintained by Gitea. Gitea's internal database (SQLite for development, PostgreSQL for production) stores all metadata including user accounts, repository details, issues, pull requests, and access tokens.

Azure Networking services including Virtual Network (VNet), Subnets, Public IP, and Network Security Groups (NSG) are configured to ensure secure communication, traffic filtering, and controlled access to the Git service. SSH (Port 22/2222) and HTTPS (Port 443) are the primary access protocols.

Azure Network Watcher is enabled for the deployment region to provide ongoing network diagnostics, connection monitoring, and traffic flow analysis.

Non-Functional Requirements

Performance: The platform targets Git operation response times below 500ms for standard repository operations and web UI page loads under 3 seconds, leveraging Gitea's lightweight Go-based runtime.

Scalability: The Azure VM can be vertically scaled to higher tiers as repository count and user load grows. Gitea supports both SQLite (lightweight) and PostgreSQL (production-grade) backends for flexible scaling.

Availability: The system is built on Azure infrastructure with 99.9% uptime SLA. Gitea and Nginx are configured as systemd services for automatic restart on failure.

Security: Security is enforced through SSH key-based authentication, HTTPS with SSL/TLS, Gitea's built-in access token system, NSG-based traffic control, and Azure Network Watcher monitoring.

Cost Efficiency: By leveraging the Azure Student Subscription, the project targets a monthly budget of Rs. 2,000-2,500, covering the VM, Public IP, and storage — a fraction of equivalent commercial Git hosting costs.

2.2 PROPOSED SOLUTION OVERVIEW

The proposed solution deploys Gitea, a lightweight self-hosted Git service, on an Azure Virtual Machine to deliver a complete Git hosting platform. Gitea is selected for its minimal resource requirements, comprehensive feature set comparable to GitHub, and ease of deployment on Linux servers.

The architecture follows a single-tier deployment model where Gitea, Nginx, and the database all run on the same Azure VM. Nginx acts as a reverse proxy handling HTTPS termination and

forwarding requests to Gitea's internal HTTP server. This design is appropriate for academic and small-team deployments where simplicity and cost efficiency are prioritized.

Gitea's built-in user management system handles authentication, organization creation, team management, and repository permission assignment. Administrators access a dedicated admin panel for system configuration, user management, and maintenance tasks such as Git repository garbage collection.

2.3 CLOUD DEPLOYMENT STRATEGY

The deployment strategy leverages Microsoft Azure as the primary cloud provider, utilizing an Infrastructure-as-a-Service (IaaS) model through Azure Virtual Machines to maintain full control over the application runtime environment.

The application is deployed on an Azure Linux Virtual Machine running Ubuntu Server 22.04 LTS. Gitea is installed as a systemd service and configured to run on an internal port, with Nginx configured as a reverse proxy for HTTPS access. A Let's Encrypt SSL certificate is provisioned via Certbot for encrypted communication.

Azure Networking is configured with a Virtual Network (VNet) and dedicated subnet for the application tier. A static Public IP address is assigned to the VM for consistent DNS configuration. Network Security Groups are configured with strict inbound and outbound rules permitting only HTTP (80), HTTPS (443), SSH (22), and Gitea SSH (2222).

Azure Network Watcher is enabled for the deployment region to provide ongoing network diagnostics, connection monitoring, and traffic flow analysis, ensuring infrastructure reliability and security visibility.

2.4 INFRASTRUCTURE REQUIREMENTS

Compute Resources:

- Azure Virtual Machine: Standard B2s (2 vCPUs, 4 GiB RAM) running Ubuntu Server 22.04 LTS, hosting the Gitea application, Nginx reverse proxy, and SQLite/PostgreSQL database.
- Operating System: Ubuntu Server 22.04 LTS for stability, long-term support, security updates, and compatibility with Gitea's Go runtime.

Storage Requirements:

- Azure VM OS Disk: 64 GB Standard SSD for the operating system, Gitea binary, configuration, and Git repository storage.
- Data Disk (Optional): Additional managed disk for high-volume repository storage, mountable and expandable without VM downtime.

2.5 AZURE SERVICES MAPPING AND JUSTIFICATION

Table 3. Azure Services Mapping

Requirement	Azure Service	Purpose	Est. Monthly Cost (Rs.)
Application Hosting	Azure VM (B2s)	Host Gitea + Nginx + PostgreSQL	2,000

Networking	Azure VNet & NSG	Secure traffic filtering and subnet management	Free
Monitoring	Azure Network Watcher	Network diagnostics and connection monitoring	Free
Public Access	Azure Public IP (Static)	Enable internet access to Git service	150
SSL Certificate	Let's Encrypt (Certbot)	Free HTTPS certificate for encrypted access	Free

CHAPTER 3 - IMPLEMENTATION

3.1 GITEA INSTALLATION AND CONFIGURATION MODULE

The Gitea installation and configuration module forms the foundation of the ForgeGit platform. Gitea is deployed directly on the Azure VM using its pre-compiled Linux binary, configured as a systemd service for automatic startup and process management.

Implementation Details:

- Binary Installation: The latest Gitea release binary is downloaded from the official Gitea releases page and installed to /usr/local/bin/gitea with appropriate execute permissions.
- User and Directory Setup: A dedicated gitea system user and group are created, along with the required directory structure at /home/gitea/gitea-repositories for repository storage and /etc/gitea for configuration files.
- Configuration (app.ini): Gitea is configured via its app.ini file, specifying the domain name, HTTP port (3000 internally), SSH domain and port (2222), database connection, and repository root path.
- Nginx Reverse Proxy: Nginx is configured to proxy incoming HTTPS requests on port 443 to Gitea's internal HTTP server on port 3000, with SSL termination handled by Nginx using Let's Encrypt certificates.
- Systemd Service: Gitea is registered as a systemd service (gitea.service) to ensure automatic startup on VM boot and automatic restart on failure.

3.2 USER AND ORGANIZATION MANAGEMENT MODULE

The User and Organization Management Module provides comprehensive account and team management capabilities through Gitea's built-in web interface and admin panel.

Administrator Functions:

- Create and manage user accounts, reset passwords, and configure site-wide settings via the admin panel at /admin.
- Configure authentication sources including built-in database authentication, LDAP, and OAuth2 providers.
- Monitor active users, repository counts, and system resource usage from the admin dashboard.

Organization and Team Functions:

- Create organizations to group related repositories and manage team memberships with defined permission levels (Owner, Admin, Write, Read).
- Assign users to teams with repository-specific access control, supporting fine-grained permission management for project teams.

3.3 REPOSITORY MANAGEMENT MODULE

The Repository Management Module provides complete Git repository hosting including creation, browsing, branching, tagging, pull requests, and code review workflows.

Core Repository Features:

- Repository Creation: Users create repositories via the web UI, selecting visibility (public/private), initialization options (README, .gitignore, license), and default branch configuration.
- HTTPS and SSH Access: Repositories support both HTTPS clone URLs (https://forgegit.example.com/user/repo.git) and SSH clone URLs (git@forgegit.example.com:2222/user/repo.git), providing flexible access methods.
- Branch and Tag Management: The web interface displays all branches and tags, enables branch protection rules, and supports merge strategies including merge commits, rebase, and squash.
- Pull Requests and Code Review: Team members create pull requests for peer code review, with support for inline comments, review approvals, and merge conflict detection.
- Web-based File Editing: Files can be directly viewed, created, and edited through the browser with syntax highlighting and commit message support.

3.4 ISSUE AND MILESTONE TRACKING MODULE

The Issue and Milestone Tracking Module provides project management capabilities integrated directly with the Git repository, enabling teams to track tasks, bugs, and project milestones.

- Issue Management: Users create issues with titles, descriptions, labels, assignees, and due dates. Issues support Markdown formatting, file attachments, and comment threads.
- Labels and Milestones: Custom labels categorize issues by type (bug, feature, documentation) and priority. Milestones group issues into release targets with progress tracking.
- Cross-Reference Linking: Issues can be referenced in commit messages using keywords (Closes #1, Fixes #2) to automatically close issues when commits are merged.

3.5 WEBHOOK AND CI INTEGRATION MODULE

The Webhook and CI Integration Module enables ForgeGit to trigger external automation workflows in response to Git events, supporting modern DevOps practices.

- Webhook Configuration: Repository webhooks are configured to send HTTP POST notifications to external endpoints upon events such as push, pull request creation, issue creation, and release publication.
- Payload Delivery: Webhook payloads are delivered as JSON with event-specific data including commit details, branch information, and user identities, compatible with standard CI/CD systems.
- Gitea Actions (Beta): Gitea's built-in Actions feature, compatible with GitHub Actions workflow syntax, can be enabled to run CI workflows directly on the ForgeGit server using self-hosted runners.

3.6 BACKUP AND STORAGE MODULE

The Backup and Storage Module ensures data persistence and disaster recovery for all repository data and Gitea configuration.

- Gitea Dump: Gitea's built-in gitea dump command creates a compressed archive of all repositories, the database, attachments, and configuration files, suitable for off-site backup.

- Automated Backup Scripts: Cron jobs are configured to run Gitea dump at scheduled intervals and transfer archives to Azure Blob Storage for durable off-VM backup retention.
- Azure VM Snapshots: Azure Disk Snapshots are periodically taken of the VM's OS disk to enable full system restore in case of VM failure or data corruption.

CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

4.1 AZURE VM DEPLOYMENT CONFIGURATION

ForgeGit is hosted on an Azure Linux Virtual Machine running Ubuntu Server 22.04 LTS. The VM is provisioned within an Azure Resource Group and configured with a Standard B2s SKU (2 vCPUs, 4 GiB RAM) to handle concurrent Git operations, web UI requests, and webhook processing efficiently.

Deployment Configuration:

- Gitea Application: Gitea runs as a systemd service on port 3000 (localhost only), managed by the gitea system user with restricted file system permissions.
- Nginx Reverse Proxy: Nginx is installed and configured as a reverse proxy, handling HTTPS termination on port 443 using Let's Encrypt certificates and forwarding requests to Gitea's internal port.
- SSH Git Access: Gitea's built-in SSH server is configured on port 2222 (non-standard) to avoid conflict with the system SSH daemon on port 22, while still being accessible through NSG rules.
- Database: SQLite is used for lightweight deployments; for production, PostgreSQL is installed on the VM and accessible only from localhost for enhanced performance and reliability.
- Service Management: Gitea and Nginx are configured as systemd services ensuring automatic startup on VM reboot and restart on process failure, providing continuous availability.
- Environment Security: Gitea's app.ini configuration file is stored with restricted permissions (0600) owned by the gitea user, preventing unauthorized reading of database credentials and secret keys.

4.2 NETWORK SECURITY AND NSG RULES

Azure Network Security Groups (NSGs) are configured to enforce strict traffic control for the ForgeGit Virtual Machine. The NSG defines inbound and outbound rules governing which network traffic is permitted to reach the application.

Table 4. Configured NSG Inbound Rules

Port / Protocol	Purpose
Port 80 (HTTP)	Allow web traffic to Nginx; redirects to HTTPS
Port 443 (HTTPS)	Allow secure HTTPS traffic for Gitea web interface
Port 22 (SSH)	Restricted SSH access for administrative VM management
Port 2222 (Gitea SSH)	Allow SSH-based Git clone, push, and pull operations
All Other Ports	Denied by default — blocks unauthorized access attempts

The NSG is associated with both the VM's network interface and the subnet within the Azure Virtual Network, implementing dual-layer security. Port 5432 (PostgreSQL) is not exposed through any NSG rule, ensuring the database remains inaccessible from the public internet. SSH access on port 22 is restricted to specific administrator IP addresses where possible.

4.3 AZURE NETWORK WATCHER MONITORING

Azure Network Watcher is enabled for the deployment region and provides comprehensive monitoring, diagnostics, and traffic analysis for the ForgeGit infrastructure.

Key Network Watcher Capabilities Utilized:

- Connection Monitor: Tracks the health and latency of network connections between the ForgeGit VM and external endpoints, enabling early detection of connectivity issues affecting Git clients.
- IP Flow Verify: Validates that NSG rules are correctly allowing or denying specific traffic flows, confirming that Git SSH (port 2222), HTTPS (443), and blocked ports behave as configured.
- Network Topology: Provides a visual representation of deployed network resources including VNet, subnets, VMs, and NSGs, simplifying infrastructure documentation and review.
- Packet Capture: Enables on-demand packet capture for deep inspection of Git protocol traffic, supporting security analysis and troubleshooting of connectivity issues.
- Diagnostics Logs: Network flow logs collect and archive all inbound and outbound traffic to the VM, providing an audit trail for security compliance and anomaly detection.

These capabilities ensure the infrastructure team has full visibility into network health, can proactively identify and resolve issues, and can enforce compliance with security policies governing Git service access.

4.4 APPLICATION SECURITY FEATURES

Security is embedded across the ForgeGit deployment through multiple complementary practices at both the application and infrastructure level.

Authentication Security: Gitea supports multiple authentication methods including built-in username/password with bcrypt hashing, SSH public key authentication for Git operations, and OAuth2/LDAP integration. Two-factor authentication (TOTP) is available for user accounts.

Access Token Management: Gitea's API access token system allows users to generate scoped tokens for CI/CD integration and API automation without exposing primary account credentials. Tokens can be revoked individually at any time.

Repository Visibility Control: Repositories can be configured as public (visible to all users), private (accessible only to authorized collaborators), or internal (visible to authenticated users only), providing flexible data exposure control.

HTTPS and SSL: Nginx is configured with a Let's Encrypt SSL/TLS certificate, ensuring all web traffic between users and the ForgeGit platform is encrypted in transit. HTTP requests are automatically redirected to HTTPS.

Gitea Security Settings: The app.ini configuration includes enabled security hardening options such as disabling user registration (admin-only user creation), enabling secret key rotation, and configuring session cookie security flags (HttpOnly, Secure, SameSite).

CHAPTER 5 - RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The ForgeGit platform was successfully deployed on Microsoft Azure, fulfilling all core objectives outlined in the project scope. The Gitea-based Git hosting service is fully operational, providing repository management, user authentication, issue tracking, pull request workflows, and webhook-based event notifications.

The Gitea application, deployed on an Azure Linux Virtual Machine behind Nginx, successfully handles Git HTTP and SSH operations with web UI page loads consistently under 2 seconds. Gitea's Go-based runtime demonstrates excellent performance characteristics, handling multiple concurrent repository operations efficiently on the B2s VM.

The Azure NSG configuration was validated by testing each configured port. HTTPS (443) and Gitea SSH (2222) traffic reached the application successfully, while attempts to access PostgreSQL (5432) or other unexposed ports were blocked as configured. Azure Network Watcher confirmed proper network topology and validated all NSG flow rules through IP Flow Verify.

Repository operations including git clone, git push, and git pull over both HTTPS and SSH were tested and confirmed functional. Pull request creation, review, and merge workflows were validated across multiple test repositories and user accounts.

5.2 CHALLENGES FACED AND RESOLUTIONS

Several technical challenges were encountered during deployment. Each was analyzed and resolved through a structured approach.

SSH Port Conflict: The default Gitea SSH port (22) conflicted with the system SSH daemon. Resolution involved configuring Gitea to use port 2222 for Git SSH operations, updating the NSG to permit port 2222, and informing users to specify the custom port in their SSH configuration.

Nginx SSL Configuration: Initial HTTPS configuration encountered issues with certificate chain ordering. Resolution involved using Certbot's Nginx plugin (certbot --nginx) for automated SSL configuration, which correctly ordered the certificate chain and configured HTTP-to-HTTPS redirection.

NSG Rule Ordering: Misconfigured NSG rule priorities initially blocked Gitea SSH access while allowing system SSH. Resolution involved methodically testing each NSG rule using Azure Network Watcher's IP Flow Verify tool and adjusting rule priority numbers to ensure correct traffic filtering order.

Gitea File Permission Issues: Gitea failed to start after installation due to incorrect directory ownership. Resolution involved ensuring all Gitea data directories (/home/gitea, /etc/gitea) were owned by the gitea system user and that the app.ini file had restricted permissions (0600) for security.

5.3 PERFORMANCE OBSERVATIONS

Performance testing confirmed the efficiency of Gitea's lightweight Go-based architecture and the Azure B2s VM configuration.

Web UI response times averaged under 200ms for dashboard, repository, and profile pages under normal load. Git HTTP clone operations for repositories under 50MB completed in under

5 seconds on the Azure infrastructure, well within acceptable thresholds for development workflows.

Gitea's memory footprint on the Azure VM remained consistently under 150MB during normal operation, leaving ample headroom on the B2s VM's 4GB RAM for additional concurrent operations. CPU utilization stayed below 15% during standard repository browsing and Git push/pull operations.

Monthly operational cost was tracked through Azure Cost Management. The Azure VM (B2s) and static Public IP combined to approximately Rs. 2,150 per month, keeping the total well within the Rs. 3,000 budget target — demonstrating exceptional cost efficiency compared to equivalent commercial Git hosting subscriptions.

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

Throughout the development and deployment of ForgeGit, the team gained substantial hands-on experience across cloud infrastructure deployment and DevOps practices.

The most significant learning was understanding Azure cloud infrastructure configuration in practice — provisioning Virtual Machines, configuring Virtual Networks and subnets, managing NSG rules with correct priority ordering, and utilizing Network Watcher for diagnostics.

Working with Gitea provided deep insight into how Git hosting platforms operate internally — from repository storage on disk to webhook delivery mechanisms and SSH key-based authentication. Understanding the full stack from Git protocol to web application layer was a valuable learning outcome.

In terms of teamwork, the group followed a collaborative approach with clearly divided responsibilities — Sanjai Kumaran focused on Azure infrastructure and networking, Monesh GJ led Gitea configuration and security hardening, and Dhanush M handled documentation, testing, and backup configuration.

CHAPTER 6 - CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

This project successfully delivered ForgeGit — a secure, scalable, and fully functional Git-as-a-Service platform deployed on Microsoft Azure using Gitea. The system addresses the limitations of commercial Git hosting by providing a self-hosted, cost-controlled, and fully customizable Git service that institutions and development teams can deploy on their own cloud infrastructure.

The technical implementation demonstrates practical application of cloud deployment, network security configuration, and DevOps infrastructure management. The Gitea application deployed on an Azure Linux Virtual Machine with Nginx provides reliable Git hosting with consistent sub-200ms web UI response times and efficient Git protocol performance. The PostgreSQL database ensures reliable metadata storage for all user accounts, repositories, and collaboration data.

From a security perspective, the combination of Azure NSG rules, Let's Encrypt SSL/TLS, Gitea's built-in access control, SSH key authentication, and Azure Network Watcher monitoring created a robust, multi-layered security posture. Network Watcher provided full infrastructure visibility and validated that all traffic filtering rules functioned as intended.

The total monthly operational cost of approximately Rs. 2,150 demonstrates that a fully featured, self-hosted Git service can be deployed at a fraction of the cost of commercial alternatives, making it immediately viable for academic and small-team deployments. The project successfully demonstrates that modern cloud technologies and open-source tools can be combined to deliver production-grade DevOps infrastructure.

6.2 FUTURE WORKS

The current implementation provides a solid and extensible foundation for numerous planned enhancements.

- Gitea Actions CI/CD: Enable and configure Gitea's built-in Actions feature (compatible with GitHub Actions workflow syntax) to provide native CI/CD pipeline execution directly within the ForgeGit platform.
- Containerized Deployment: Migrate from VM-based deployment to Docker containers orchestrated with Docker Compose or Azure Container Apps, improving deployment consistency and simplifying scaling.
- Azure Blob Storage Integration: Configure Gitea's LFS (Large File Storage) to use Azure Blob Storage as the backend, enabling efficient storage of large binary files outside the VM disk.
- High Availability Setup: Deploy a secondary Azure VM with Gitea in a failover configuration using Azure Load Balancer, improving availability for production environments.
- LDAP/Active Directory Integration: Integrate Gitea with institutional LDAP or Azure Active Directory for centralized user authentication, enabling single sign-on across institutional systems.
- Automated Backup to Azure Blob: Implement scheduled Gitea dump backups uploaded automatically to Azure Blob Storage with retention policies for disaster recovery.

- Analytics and Monitoring Dashboard: Integrate Prometheus metrics exposed by Gitea with Grafana dashboards for real-time monitoring of repository activity, user engagement, and system performance.
- Mirror Repositories: Configure Gitea's repository mirroring feature to periodically synchronize selected repositories from GitHub or GitLab, enabling backup and offline access to critical external dependencies.

REFERENCES

1. Gitea Documentation. (2024). Gitea - Git with a cup of tea. https://docs.gitea.com
2. Microsoft Azure. (2024). Azure Virtual Machines Documentation. https://docs.microsoft.com/en-us/azure/virtual-machines/
3. Microsoft Azure. (2024). Azure Network Security Groups. https://docs.microsoft.com/en-us/azure/virtual-network/network-security-groups-overview
4. Microsoft Azure. (2024). Azure Network Watcher Documentation. https://docs.microsoft.com/en-us/azure/network-watcher/
5. Nginx. (2024). Nginx Reverse Proxy Documentation. https://nginx.org/en/docs/
6. Let's Encrypt / EFF. (2024). Certbot Documentation. https://certbot.eff.org/docs/
7. PostgreSQL Global Development Group. (2024). PostgreSQL Documentation. https://www.postgresql.org/docs/
8. Git SCM. (2024). Git Reference Documentation. https://git-scm.com/docs